

AI-Powered **SOAR** Pipeline:

Wazuh + n8n + AI Integration

AI-Powered SOAR Pipeline: Wazuh + n8n + AI Integration

Performed by: Muhammad Shayan Younas

Summary

This project demonstrates an end-to-end AI-driven Security Orchestration, Automation, and Response (SOAR) pipeline built by integrating Wazuh (SIEM/detection engine), n8n (workflow automation platform), and Google's Gemini AI (intelligent alert analysis). Rather than simply forwarding raw security alerts, the pipeline actively interprets each alert's content, classifies its severity (High/Medium/Low), generates a plain-English explanation, and recommends a concrete response action — before automatically routing a formatted notification email to the security analyst based on that severity level.

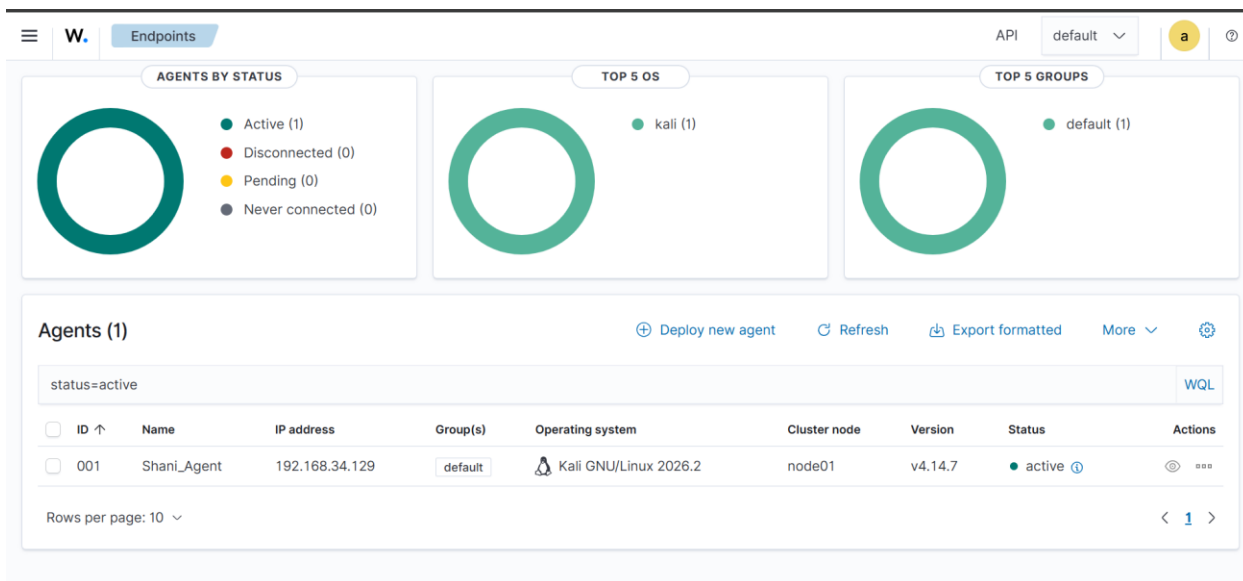
The system was built from the ground up on virtual machines (Kali Linux as the monitored endpoint/attacker, Ubuntu Server hosting the Wazuh manager), with a custom Python integration script forwarding real-time Wazuh alerts to an n8n webhook. There, an HTTP Request node sends the alert data to Gemini for analysis, a Code node parses the AI's structured JSON response, and a two-tier If/If1 branching structure routes the result to one of three Gmail notification nodes.

The pipeline was validated using both manually crafted test payloads (via curl) for all three severity branches, and a genuine SSH brute-force attack launched from the Kali VM,

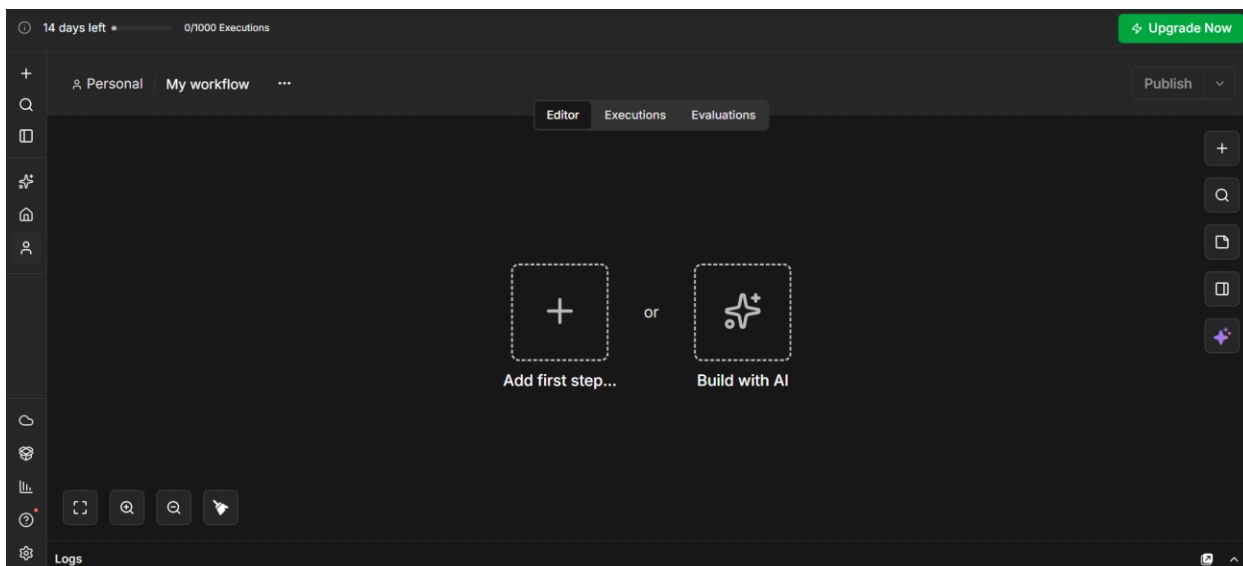
confirming that the system can detect, analyze, and respond to real attack patterns automatically and end to end — moving beyond passive alerting toward practical, AI-assisted SOC triage.

Walkthrough

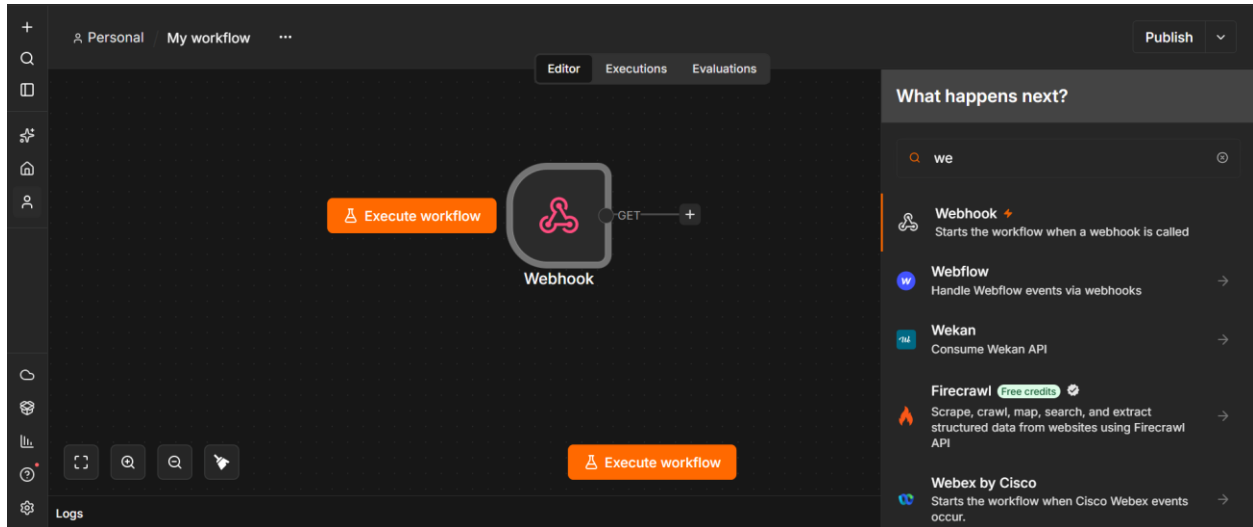
Wazuh Dashboard: Kali Agent Enrolled and Active



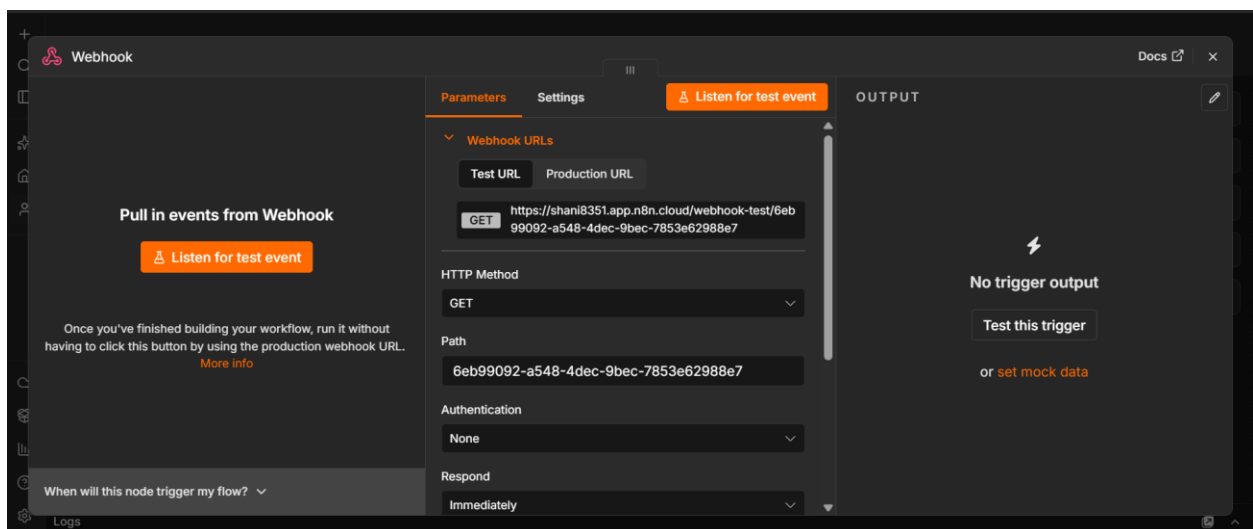
n8n: New Empty Workflow Canvas



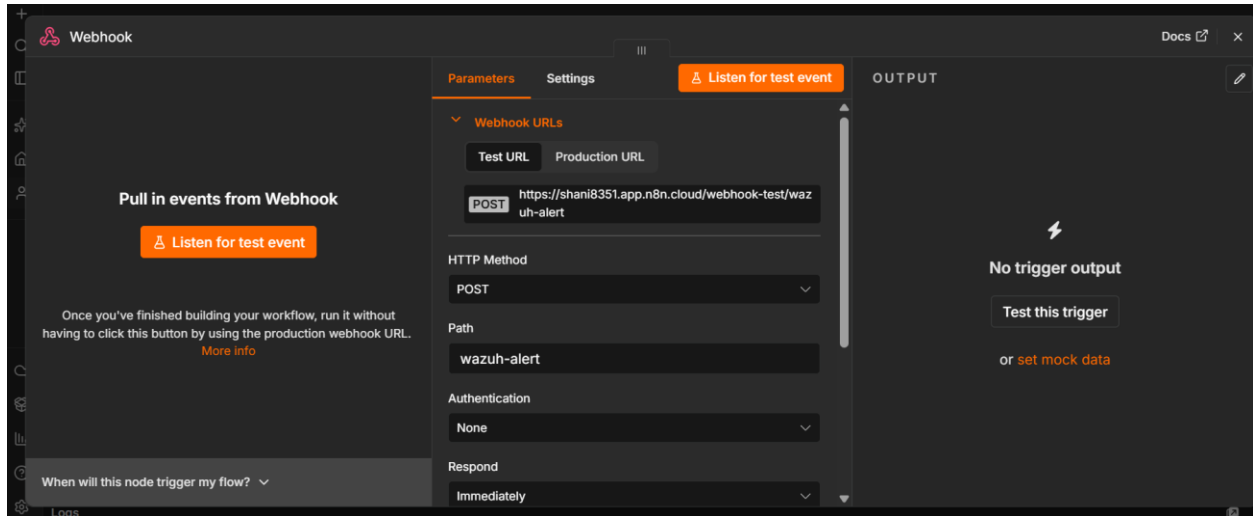
Adding the Webhook Trigger Node



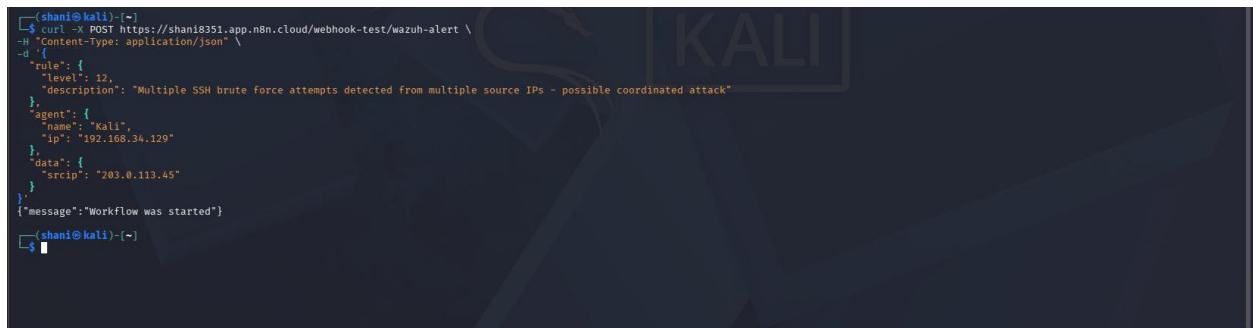
Webhook Node: Default GET Configuration



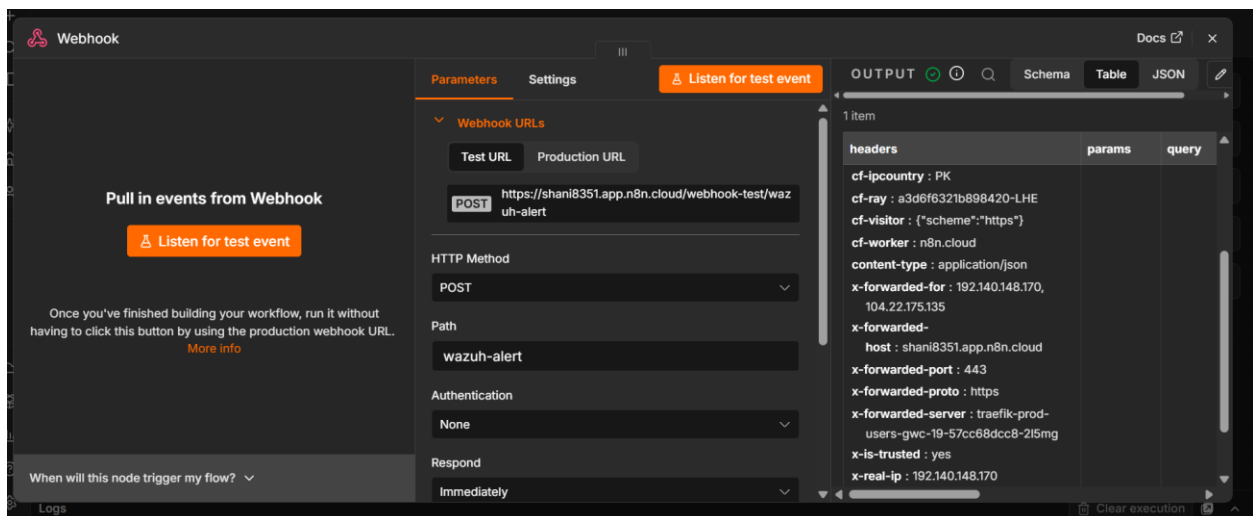
Webhook Node: Configured for POST on /wazuh-alert



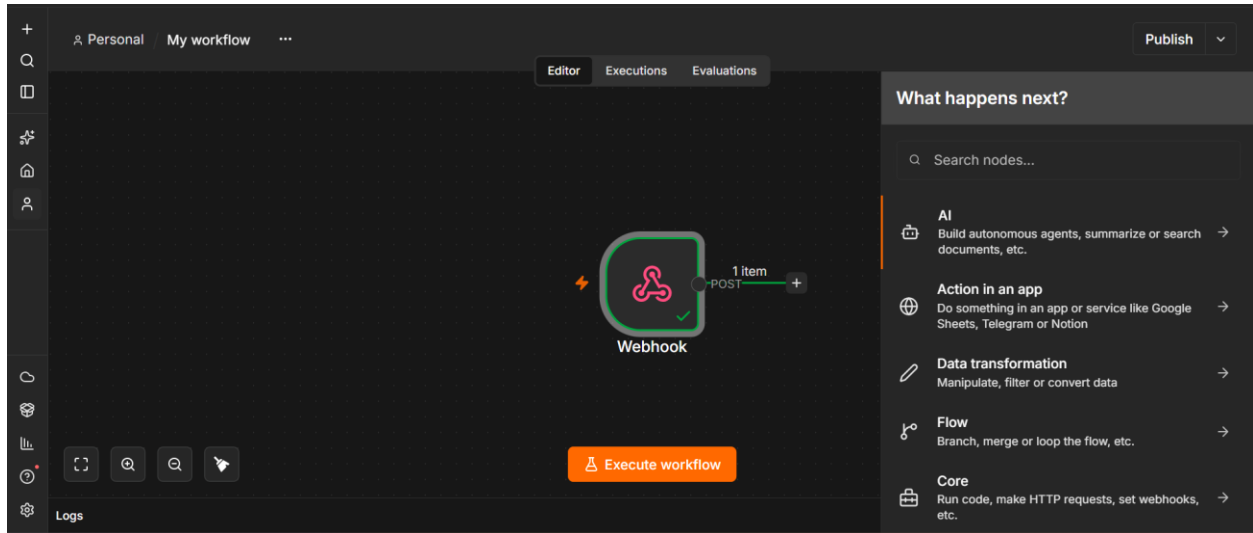
Manual Test: Sending a High-Severity Alert via curl



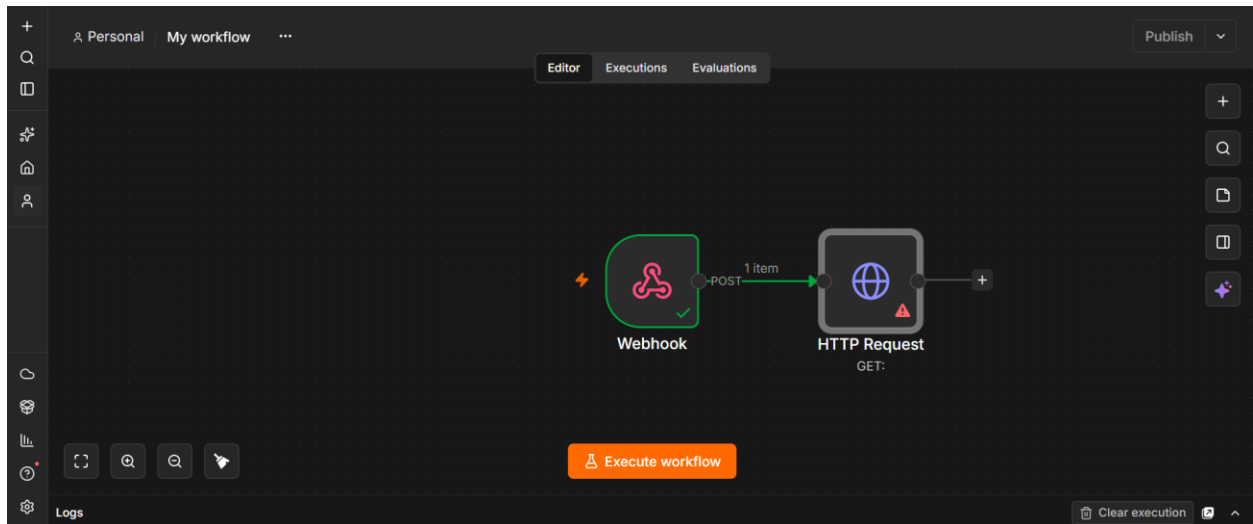
Webhook Node: Captured Incoming Alert Headers



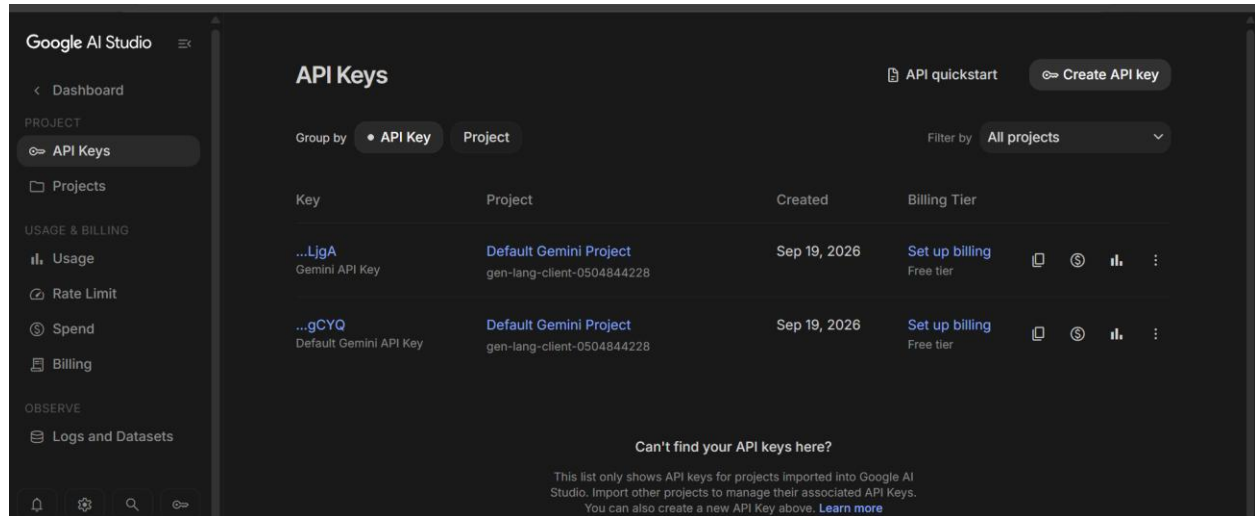
Webhook Execution Successful: Adding the Next Node



Adding the HTTP Request Node (Gemini API Call)



Generating the Gemini API Key in Google AI Studio



Google AI Studio

API Keys

Group by: API Key Project

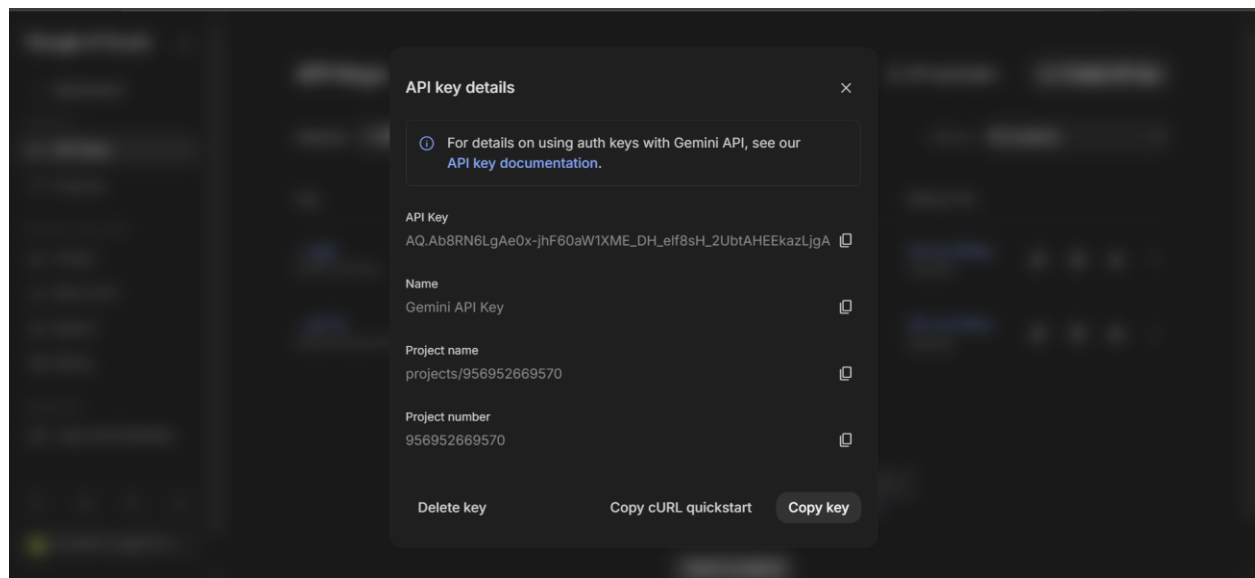
Filter by: All projects

Key	Project	Created	Billing Tier
...LjgA Gemini API Key	Default Gemini Project gen-lang-client-0504844228	Sep 19, 2026	Set up billing Free tier
...gCYQ Default Gemini API Key	Default Gemini Project gen-lang-client-0504844228	Sep 19, 2026	Set up billing Free tier

Can't find your API keys here?

This list only shows API keys for projects imported into Google AI Studio. Import other projects to manage their associated API Keys. You can also create a new API Key above. [Learn more](#)

Gemini API Key Details



API key details

For details on using auth keys with Gemini API, see our [API key documentation](#).

API Key
AQ.Ab8RN6LgAe0x-jhF60aW1XME_DH_elf8sH_2UbtAHEEkazLjgA

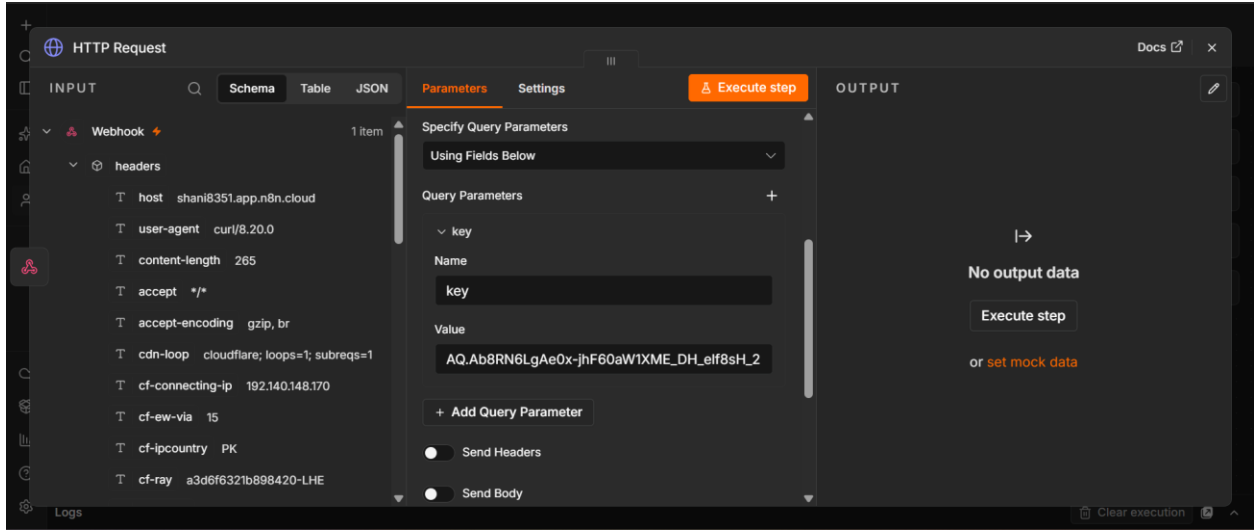
Name
Gemini API Key

Project name
projects/956952669570

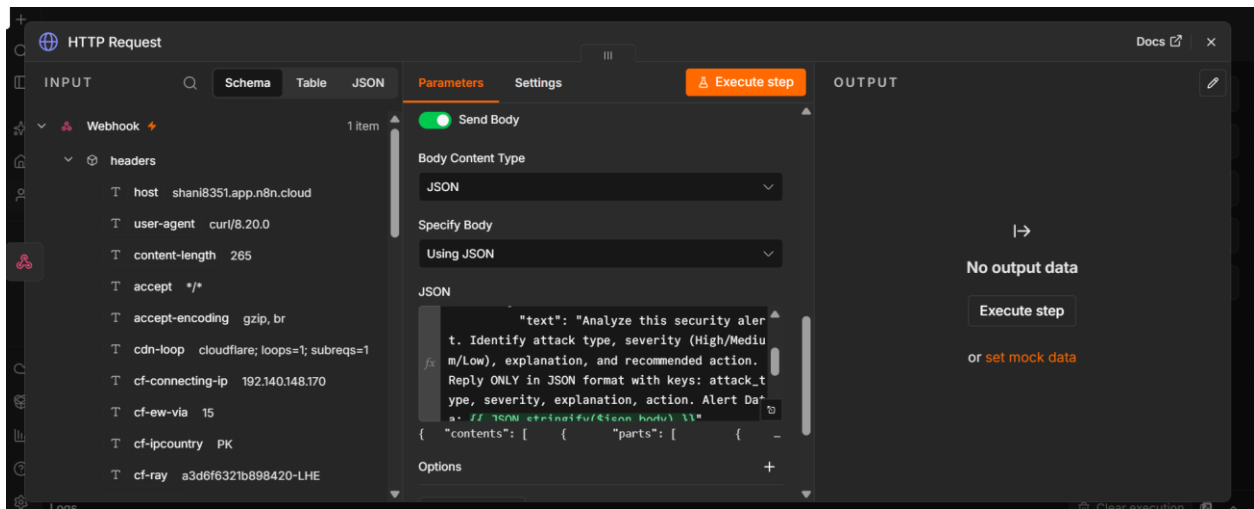
Project number
956952669570

Delete key Copy cURL quickstart Copy key

HTTP Request Node: API Key as Query Parameter



HTTP Request Node: JSON Body with Analysis Prompt



HTTP Request Node: Successful Gemini Response (High Severity)

The screenshot shows the configuration and execution of an HTTP Request node. The left pane displays the input headers for a Webhook. The center pane shows the JSON body being sent, which is a JSON.stringify call to a function. The right pane shows the output, which is a JSON response from Gemini. A notification at the bottom right indicates the node executed successfully.

INPUT

Webhook

headers

- host: shani8351.app.n8n.cloud
- user-agent: curl/8.20.0
- content-length: 265
- accept: */*
- accept-encoding: gzip, br
- cdn-loop: cloudflare, loops=1, subreqs=1
- cf-connecting-ip: 192.140.148.170
- cf-ew-via: 15
- cf-ipcountry: PK
- cf-ray: a3d6f6321b898420-LHE

JSON

```
{JSON.stringify({
  contents: [
    {
      parts: [
        {
          text: "Analyze this security alert"
        }
      ]
    }
  ]
})}
```

OUTPUT

1 item

candidates

0

content

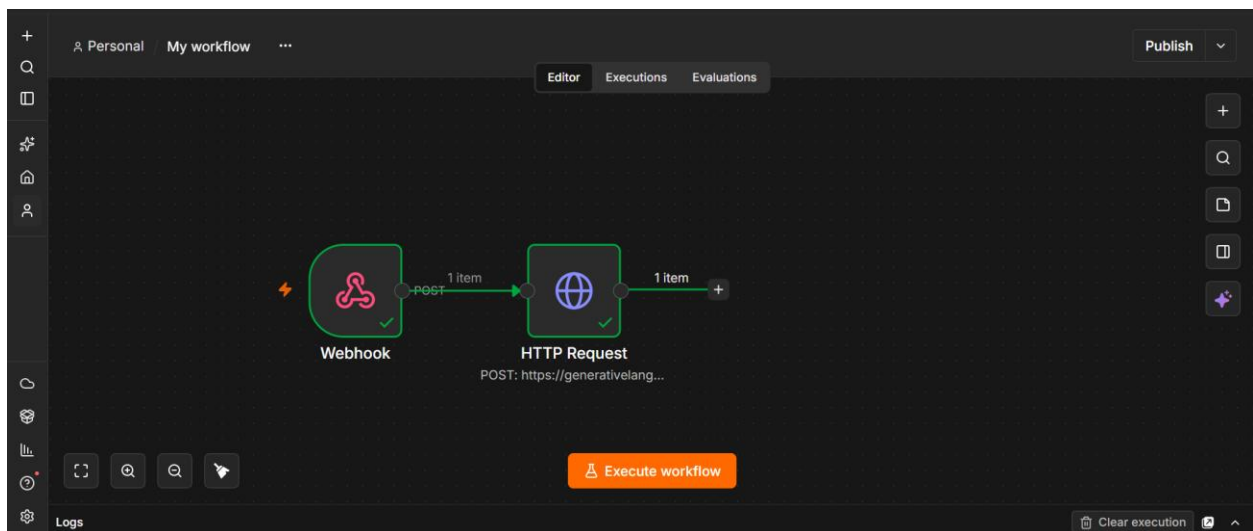
parts

0

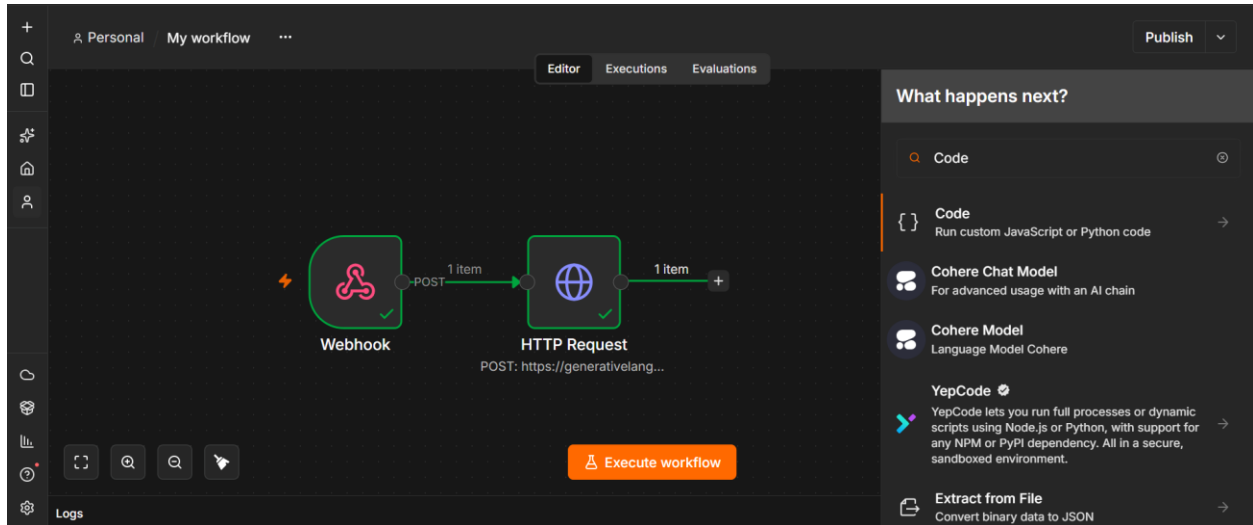
text: {\n "attack_type": "SSH Brute Force Attack",\n "severity": "High",\n "explanation": "The system detected a high volume of failed SSH login attempts originating from an external source IP (203.0.113.45) targeting the host 'Kali' (192.168.34.129). The pattern suggests a coordinated distributed brute-force attack aiming to gain unauthorized access via credential guessing.",\n "action": "Block the C"}

Node executed successfully

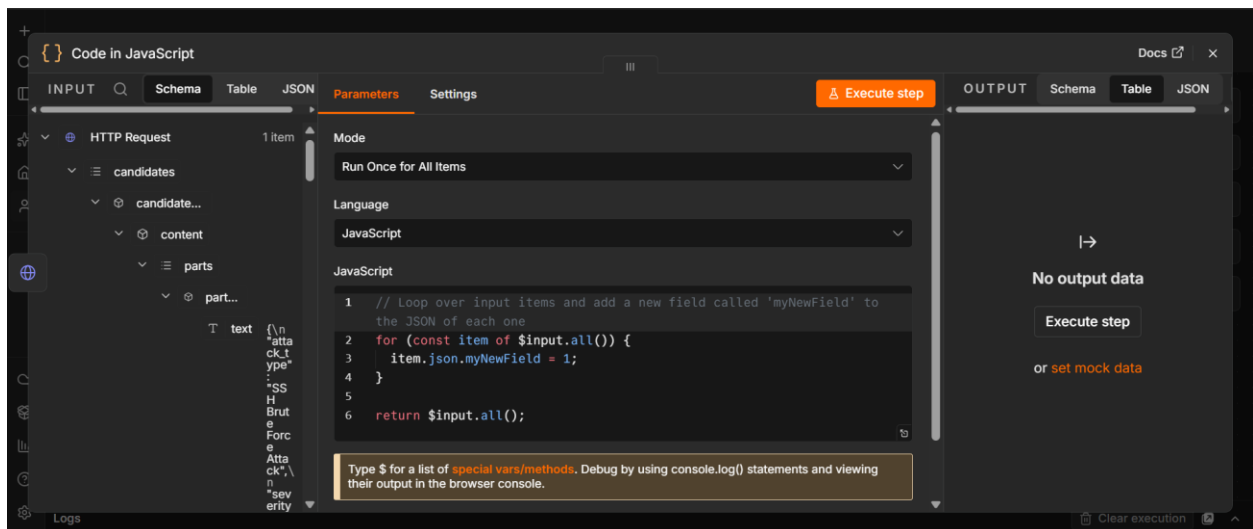
Webhook to HTTP Request Chain Executing Successfully



Adding the Code Node to Parse Gemini's Response



Code Node: Default Template



Code Node: Custom JavaScript to Parse AI Output

Code in JavaScript

INPUT | Schema | Table | JSON | Parameters | Settings | Execute step | OUTPUT | Schema | Table | JSON

Language: JavaScript

```
JavaScript
1 const rawText =
2   $input.first().json.candidates[0].content.parts[0].text;
3 const cleanText = rawText.replace(/```json```/g, "").trim();
4 const aiResult = JSON.parse(cleanText);
5
6 return {
7   json: {
8     attack_type: aiResult.attack_type,
9     severity: aiResult.severity,
10    explanation: aiResult.explanation,
11    action: aiResult.action
12  }
13 }
```

No output data

Execute step or set mock data

Code Node: Parsed Output (attack_type, severity, explanation, action)

Code in JavaScript

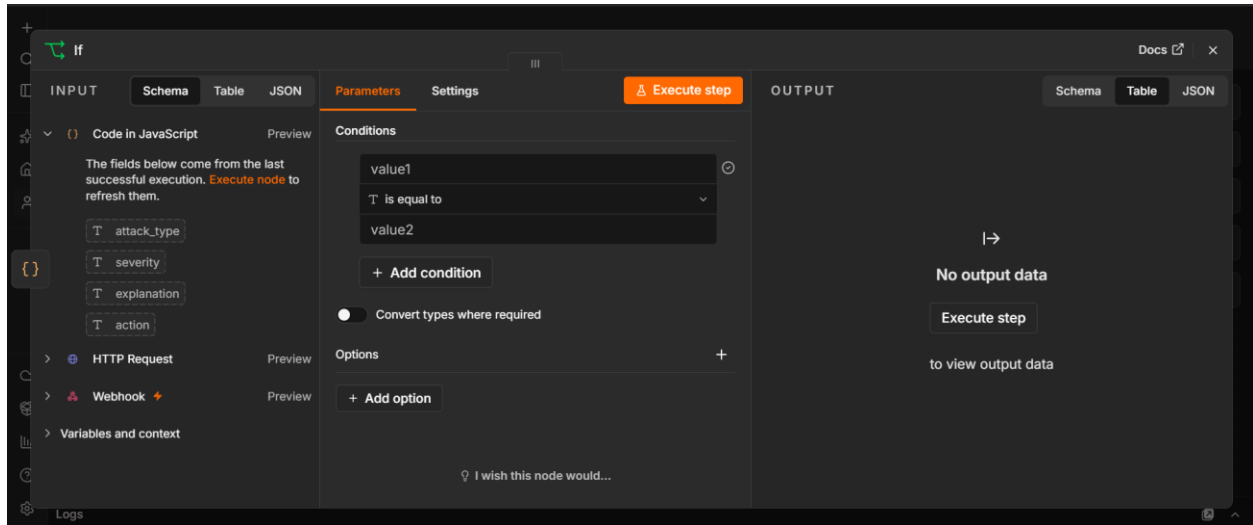
INPUT | Schema | Table | JSON | Parameters | Settings | Execute step | OUTPUT | Schema | Table | JSON

Language: JavaScript

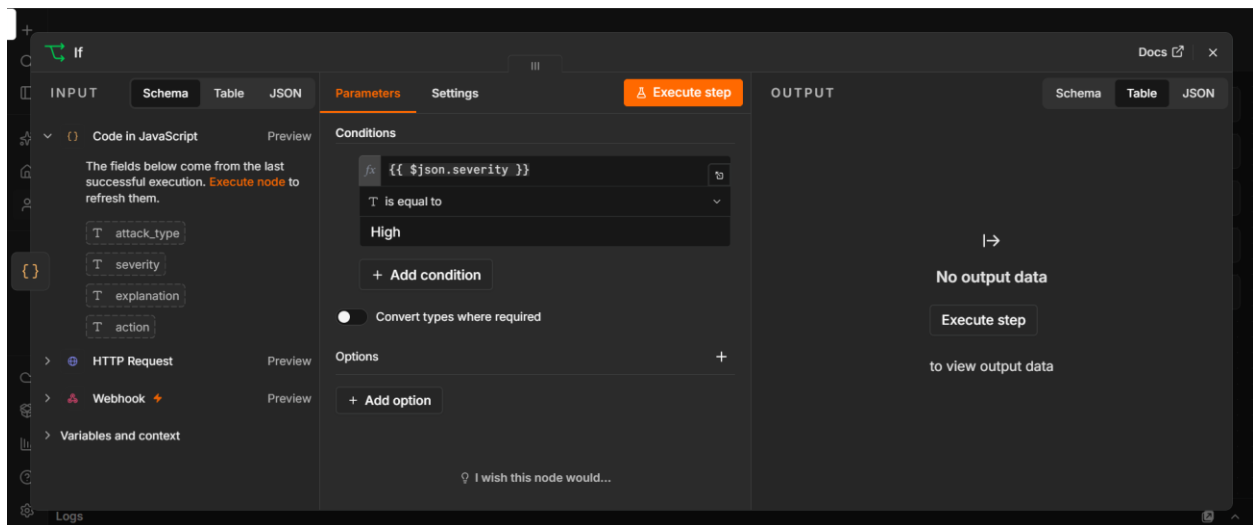
```
JavaScript
1 const rawText =
2   $input.first().json.candidates[0].content.
3   parts[0].text;
4 const cleanText =
5   rawText.replace(/```json```/g,
6   "").trim();
7 const aiResult = JSON.parse(cleanText);
8
9 return {
10   json: {
11     attack_type: aiResult.attack_type,
12     severity: aiResult.severity,
13     explanation: aiResult.explanation,
14     action: aiResult.action
15   }
16 }
```

attack_type	severity	explanation	action
SSH Brute Force Attack	High	The system detected a high volume of failed SSH login attempts originating from an external source IP (203.0.113.45) targeting the host 'Kali' (192.168.34.129). The pattern suggests a coordinated distributed brute-force attack aiming to gain unauthorized access via credential	Block the offending source IP address (203.0.113.45) on the edge firewall or via host-based controls (e.g., iptables/fail2ban), check authentication logs to confirm no successful logins occurred from this IP, and enforce SSH hardening measures such as key-based authentication and disabling root login.

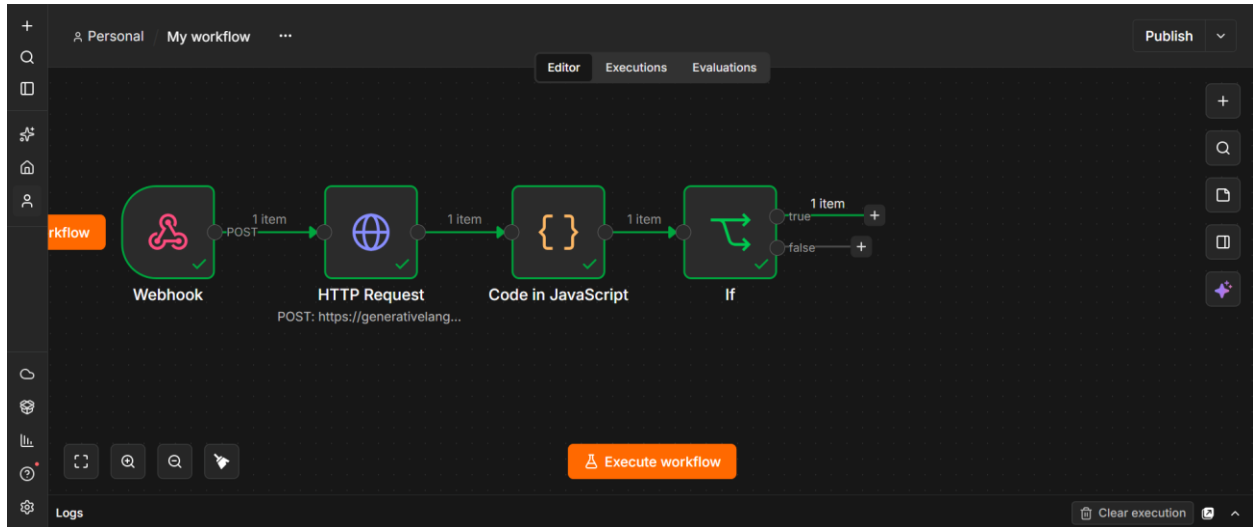
Adding the First If Node (Severity Branching)



If Node: Condition Set for “High” Severity



Full Pipeline Executing Successfully Through the If Node



Configuring the Gmail Node for High-Severity Alerts

The screenshot shows the configuration for the 'Send a message' node in n8n. The 'INPUT' section on the left lists the data being passed to the node:

- attack_type:** Distributed SSH Brute Force
- severity:** High
- explanation:** A high-severity rule (Level 12) was triggered indicating a coordinated SSH brute force attack targeting the system 'Kali' (192.168.34.129) from multiple source IPs, including 203.0.113.45, attempting to gain unauthorized remote access.
- action:** Immediately block the offending source IP address (203.0.113.45) at the firewall, verify if any login attempts succeeded, enforce public-key authentication for SSH, disable password authentication, and implement an automated IP banning tool like Fail2ban.

The 'Parameters' section on the right is configured for sending an email via Gmail:

- Credential:** Selected as 'Gmail account' (Gmail OAuth2 API).
- Send:** Set to 'Send'.
- To:** info@example.com
- Subject:** Hello World!
- Email Type:** HTML

The 'OUTPUT' section on the right shows 'No output data' and an 'Execute step' button. The bottom status bar includes 'Logs' and a 'Clear execution' button.

Gmail (High) Node: Recipient and Subject Configured

The screenshot shows the configuration for the 'Send a message' node. The 'Parameters' tab is selected, displaying the following fields:

- To:** moueez8351@gmail.com
- Subject:** HIGH Severity Alert Detected
- Email Type:** HTML
- Message:** (Empty text area)
- Options:** + Add option

The 'INPUT' section on the left shows a conditional logic block with the following fields:

- attack_type:** Distributed SSH Brute Force
- severity:** High
- explanation:** A high-severity rule (Level 12) was triggered indicating a coordinated SSH brute force attack targeting the system 'kali' (192.168.34.129) from multiple source IPs, including 203.0.113.45, attempting to gain unauthorized remote access.
- action:** Immediately block the offending source IP address (203.0.113.45) at the firewall, verify if any login attempts succeeded, enforce public-key authentication for SSH, disable password authentication, and implement an automated IP banning tool like Fail2ban.

The 'OUTPUT' section on the right shows 'No output data' and an 'Execute step' button.

Gmail (High) Node: Message Body Template

The screenshot shows the configuration for the 'Send a message' node. The 'Parameters' tab is selected, displaying the following fields:

- Subject:** HIGH Severity Alert Detected
- Email Type:** HTML
- Message:** Attack Type: {{ \$json.attack_type }} Severity: {{ \$json.severity }} Explanation: {{ \$json.explanation }} Recommended Action: {{ \$json.action }}
- Options:** + Add option

The 'INPUT' section on the left shows a conditional logic block with the following fields:

- attack_type:** Distributed SSH Brute Force
- severity:** High
- explanation:** A high-severity rule (Level 12) was triggered indicating a coordinated SSH brute force attack targeting the system 'kali' (192.168.34.129) from multiple source IPs, including 203.0.113.45, attempting to gain unauthorized remote access.
- action:** Immediately block the offending source IP address (203.0.113.45) at the firewall, verify if any login attempts succeeded, enforce public-key authentication for SSH, disable password authentication, and implement an automated IP banning tool like Fail2ban.

The 'OUTPUT' section on the right shows 'No output data' and an 'Execute step' button.

Gmail (High) Node: Executed Successfully

The screenshot shows a workflow node titled "Send a message" in a dark-themed editor. The node is configured with an "If" condition and an "action" step. The "If" condition is set to "attack_type" with the value "Distributed SSH Brute Force" and "severity" with the value "High". The "action" step is "Immediately block the offending source IP address (203.0.113.45) at the firewall, verify if any login attempts succeeded, enforce public-key authentication for SSH, disable password authentication, and implement an automated IP banning tool like Fail2ban." The "Message" field is populated with a template: "Attack Type: {{ \$json.attack_type }} Severity: {{ \$json.severity }} Explanation: {{ \$json.explanation }} Recommended Action: {{ \$json.action }}". The "Output" tab shows a table with 1 item, containing columns "id", "threadid", and "labels". The "id" is "1a0b8fb13f4494b6", "threadid" is "1a0b8fb13f4494b6", and "labels" are "0 : UNREAD", "1 : SENT", and "2 : INBOX". A green checkmark and the text "Node executed successfully" are displayed at the bottom right of the node.

id	threadid	labels
1a0b8fb13f4494b6	1a0b8fb13f4494b6	0 : UNREAD 1 : SENT 2 : INBOX

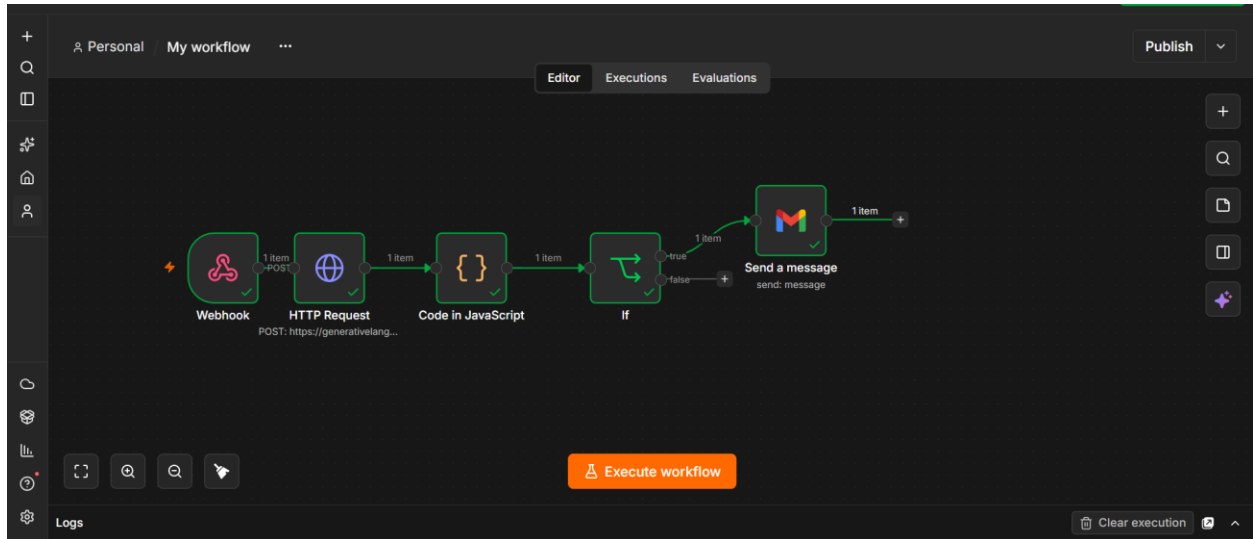
Received Email: HIGH Severity Alert Detected

The screenshot shows a Gmail inbox with a search bar at the top. The email titled "HIGH Severity Alert Detected" is selected. The email is from "moueez khan" (moueez8351@gmail.com) and was received 14:24 (0 minutes ago). The email content is as follows:

Attack Type: Distributed SSH Brute Force Severity: High Explanation: A high-severity rule (Level 12) was triggered indicating a coordinated SSH brute force attack targeting the system 'Kali' (192.168.34.129) from multiple source IPs, including 203.0.113.45, attempting to gain unauthorized remote access. Recommended Action: Immediately block the offending source IP address (203.0.113.45) at the firewall, verify if any login attempts succeeded, enforce public-key authentication for SSH, disable password authentication, and implement an automated IP banning tool like Fail2ban.

This email was sent automatically with n8n

High-Severity Branch Validated End-to-End



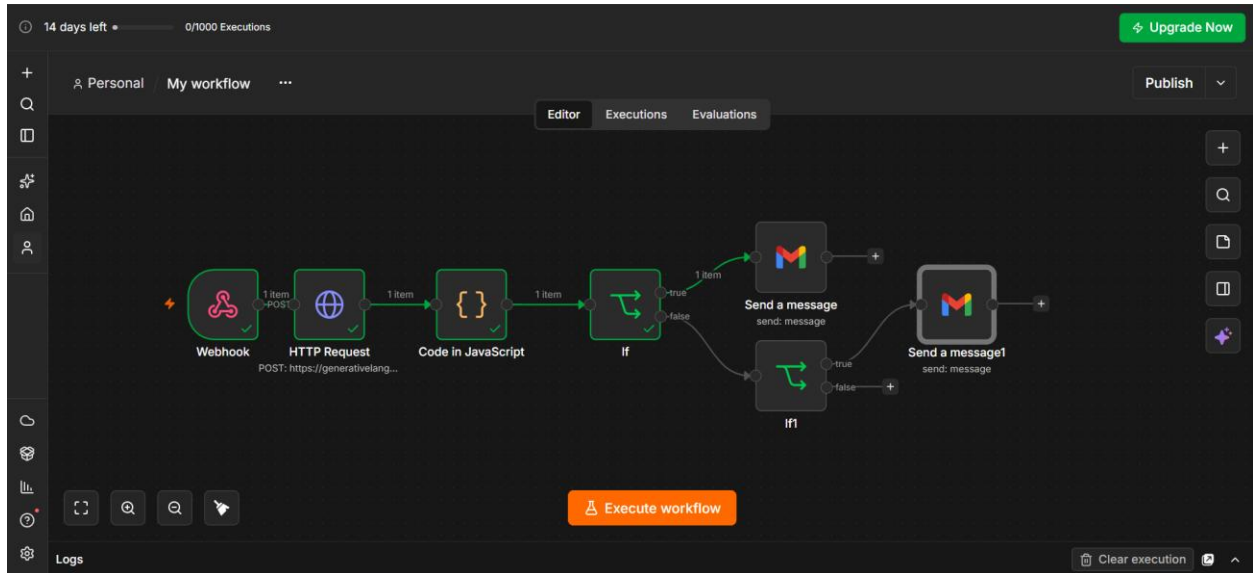
Configuring the Gmail Node for Medium-Severity Alerts

The screenshot shows the configuration for the 'Send a message' node in the workflow editor. The 'Parameters' tab is active, displaying the following settings: 1. 'Email Type' is set to 'HTML'. 2. 'Message' is a JSON template:

```
MEDIUM SEVERITY SECURITY ALERT Attack Type: {{ $json.attack_type }} Severity: {{ $json.severity }} Explanation: {{ $json.explanation }} Recommended Action: {{ $json.action }}
```

 3. 'Options' is empty, with a '+ Add option' button. The 'OUTPUT' section on the right shows 'No output data' and an 'Execute step' button. The 'INPUT' section on the left shows the 'If' node with 'Execute previous nodes to view input data'.

Two-Tier If/If1 Branching Structure Complete



Manual Test: Sending a Medium-Severity Alert via curl

```
(shani@kali)~$ curl -X POST https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert \
-H "Content-Type: application/json" \
-d '{
  "rule": {
    "level": 5,
    "description": "Suspicious sudo command executed by non-admin user - possible privilege escalation attempt"
  },
  "agent": {
    "name": "Kali",
    "ip": "192.168.34.129"
  },
  "data": {
    "srcip": "192.168.1.42"
  }
}'
{"message": "Workflow was started"}
(shani@kali)~$
```

Webhook Node: Captured Medium-Severity Payload

The screenshot shows the Webhook node interface. On the left, a panel indicates "Listening for test event" with a URL `https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert` and a "Stop Listening" button. The main area is divided into "Parameters" and "Settings" tabs. Under "Parameters", the "Webhook URLs" section shows a "Test URL" of `https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert` with a "POST" method. The "HTTP Method" is set to "POST", the "Path" is `wazuh-alert`, and "Authentication" is set to "None". The "Respond" section is empty. On the right, the "OUTPUT" tab displays a single item with a table of headers, params, and query. The headers include `host`, `user-agent`, `content-length`, `accept`, `accept-encoding`, `cdn-loop`, `cf-connecting-ip`, `cf-ew-via`, `cf-ipcountry`, `cf-ray`, `cf-visitor`, `cf-worker`, and `content-type`.

headers	params	query
<code>host : shani8351.app.n8n.cloud</code> <code>user-agent : curl/8.20.0</code> <code>content-length : 257</code> <code>accept : */*</code> <code>accept-encoding : gzip, br</code> <code>cdn-loop : cloudflare; loops=1; subreqs=1</code> <code>cf-connecting-ip : 192.140.148.170</code> <code>cf-ew-via : 15</code> <code>cf-ipcountry : PK</code> <code>cf-ray : a3d799563b8c8420-LHE</code> <code>cf-visitor : {scheme:"https"}</code> <code>cf-worker : n8n.cloud</code> <code>content-type : application/json</code>	<code>(empty object)</code>	<code>(empty object)</code>

Webhook Output: Parsed Medium-Severity Alert Body

The screenshot shows the Webhook node interface. On the left, a panel indicates "Pull in events from Webhook" with a "Listen for test event" button. The main area is divided into "Parameters" and "Settings" tabs. Under "Parameters", the "Webhook URLs" section shows a "Test URL" of `https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert` with a "POST" method. The "HTTP Method" is set to "POST", the "Path" is `wazuh-alert`, and "Authentication" is set to "None". The "Respond" section is empty. On the right, the "OUTPUT" tab displays a single item with a table of body and webhookUrl. The body contains a rule with level 5, a description, an agent name, IP, and data.

body	webhookUrl
<code>rule</code> <code>level : 5</code> <code>description : Suspicious sudo command executed by non-admin user - possible privilege escalation attempt</code> <code>agent</code> <code>name : Kali</code> <code>ip : 192.168.34.129</code> <code>data</code> <code>srcip : 192.168.1.42</code>	<code>https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert</code>

Configuring the Gmail Node for Low-Severity Alerts

The screenshot shows the 'Send a message2' node configuration in a workflow editor. The 'Parameters' tab is active, showing the following settings:

- To:** moueez8351@gmail.com
- Subject:** LOW Severity Alert - Logged
- Email Type:** HTML
- Message:** A JavaScript code block containing a template for a low-severity security event alert. The template uses JSON path expressions like `{{ $json.attack_type }}` and `{{ $json.severity }}` to dynamically insert data from the incoming payload.

The 'OUTPUT' tab on the right shows 'No output data' and an 'Execute step' button.

Manual Test: Sending a Low-Severity Alert via curl

```
(shani@kali)-[~]
$ curl -X POST https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert \
-H "Content-Type: application/json" \
-d '{
  "rule": {
    "level": 2,
    "description": "Routine system file integrity check completed successfully - no anomalies found"
  },
  "agent": {
    "name": "Kali",
    "ip": "192.168.34.129"
  },
  "data": {
    "srcip": "192.168.1.5"
  }
}'
{"message": "Workflow was started"}
```

Webhook Node: Captured Low-Severity Payload

The screenshot shows the 'Webhook' node configuration in a workflow editor. The 'Parameters' tab is active, showing the following settings:

- HTTP Method:** POST
- Path:** wazuh-alert
- Authentication:** None
- Respond:** Immediately

The 'OUTPUT' tab on the right shows the captured payload as a single item with the following structure:

```
{
  "body": {
    "rule": {
      "level": 2,
      "description": "Routine system file integrity check completed successfully - no anomalies found"
    },
    "agent": {
      "name": "Kali",
      "ip": "192.168.34.129"
    },
    "data": {
      "srcip": "192.168.1.5"
    }
  },
  "webhookUrl": "https://shani8351.app.n8n.cloud/webhook-test/wazuh-alert"
}
```

Custom Python Integration Script on the Wazuh Manager

```
GNU nano 6.2 /var/ossec/integrations/custom-n8n *
#!/usr/bin/env python3

import sys
import json
import requests

alert_file = sys.argv[1]
webhook_url = "https://shani8351.app.n8n.cloud/webhook/wazuh-alert"

try:
    with open(alert_file) as f:
        alert_json = json.load(f)
        requests.post(webhook_url, json=alert_json, timeout=30)
except Exception as e:
    with open("/var/ossec/logs/integrations.log", "a") as log:
        log.write(f"Error: {str(e)}\n")_

Help      Write Out  Where Is  Cut       Execute   Location  Undo
Exit      Read File  Replace  Paste     Justify   Go To Line Redo
```

Wazuh ossec.conf: n8n Integration Block

```
<integration>

  <name>custom-n8n</name>

  <hook_url>https://shani8351.app.n8n.cloud/webhook/wazuh-alert</hook_url>

  <alert_format>json</alert_format>

</integration>
</ossec_config>
```

Wazuh Manager Service Status: Active and Running

```
• wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-09-19 09:55:30 UTC; 1min 35s ago
   Process: 122730 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0)
   Tasks: 251 (limit: 4510)
   Memory: 1.8G
   CPU: 39.003s
   CGroup: /system.slice/wazuh-manager.service
           └─ 55459 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              55460 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              55463 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              55466 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              55479 /var/ossec/bin/wazuh-db
              55515 /var/ossec/bin/wazuh-execd
              55575 /var/ossec/bin/wazuh-syscheckd
              122793 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              122794 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              122795 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              122798 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              122801 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              122819 /var/ossec/bin/wazuh-integrator
              122840 /var/ossec/bin/wazuh-authd
              122855 /var/ossec/bin/wazuh-db
              122892 /var/ossec/bin/wazuh-execd
              122911 /var/ossec/bin/wazuh-analysisd
              122954 /var/ossec/bin/wazuh-syscheckd
              122972 /var/ossec/bin/wazuh-remoted
              123008 /var/ossec/bin/wazuh-logcollector
              123026 /var/ossec/bin/wazuh-monitord
              123051 /var/ossec/bin/wazuh-modulesd

Sep 19 09:55:23 shani env[122790]: Started wazuh-syscheckd...
Sep 19 09:55:24 shani env[122790]: Started wazuh-remoted...
Sep 19 09:55:25 shani env[122790]: Started wazuh-logcollector...
Sep 19 09:55:26 shani env[122790]: Started wazuh-monitord...
Sep 19 09:55:26 shani env[123046]: 2026/09/19 09:55:26 wazuh-modulesd:router: INFO: Loaded router m
lines 1-36
```

Installing sshpass on Kali for the Live Attack Test

```
(shani@kali)~$
$ for i in {1..8}; do sshpass -p 'wrongpass123' ssh -o StrictHostKeyChecking=no kali@127.0.0.1; sleep 2; done
Command 'sshpass' not found, but can be installed with:
sudo apt install sshpass
Do you want to install it? (N/y)y
sudo apt install sshpass
[sudo] password for shani:
The following package was automatically installed and is no longer required:
  iucode-tool
Use 'sudo apt autoremove' to remove it.

Installing:
  sshpass

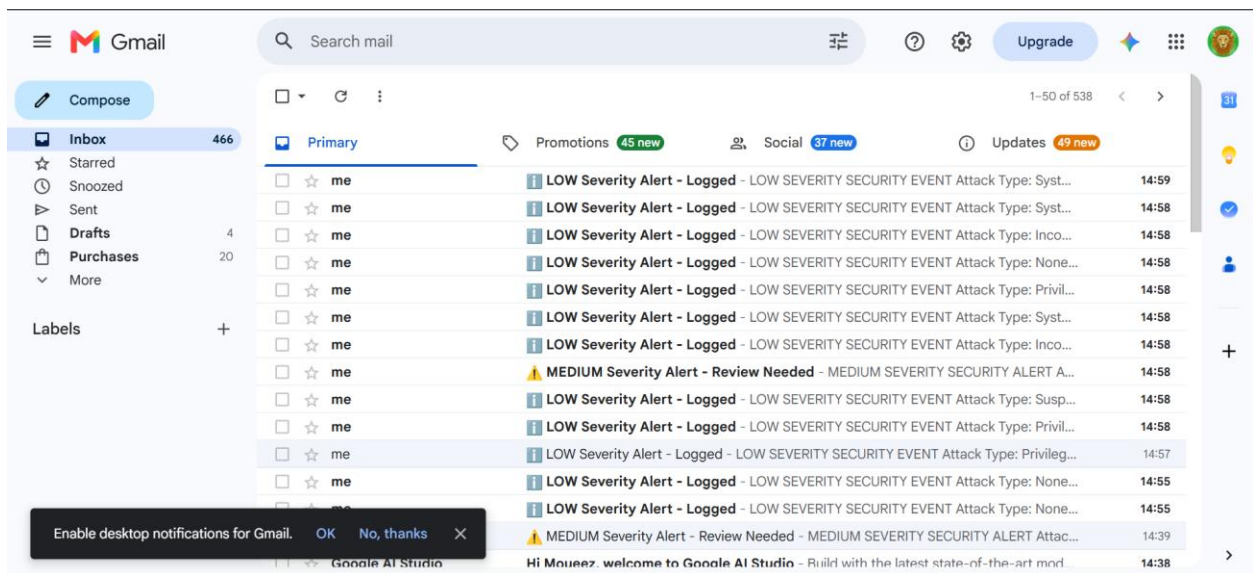
Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1232
  Download size: 12.7 kB
  Space needed: 41.0 kB / 31.3 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 sshpass amd64 1.10-0.1 [12.7 kB]
Fetched 12.7 kB in 1s (10.1 kB/s)
Selecting previously unselected package sshpass.
(Reading database... 425377 files and directories currently installed.)
Preparing to unpack ../sshpass_1.10-0.1_amd64.deb...
Unpacking sshpass (1.10-0.1)...
Setting up sshpass (1.10-0.1)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.2.6)...
```

Real SSH Brute-Force Attempts Against the Target

```
(shani@kali)-[~]
└─$ for i in {1..8}; do sshpass -p 'wrongpass123' ssh -o StrictHostKeyChecking=no kali@127.0.0.1; sleep 2; done
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
ssh: connect to host 127.0.0.1 port 22: Connection refused
(shani@kali)-[~]
```

Gmail Inbox Flooded with Real-Time Alerts from the Live Attack



Starting the SSH Service on the Kali Target

```
(shani@kali)-[~]
└─$ sudo systemctl start ssh
Kali Linux
(shani@kali)-[~]
└─$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-09-19 05:59:53 EDT; 18s ago
     Invocation: 3bd5ed924a68429eb085495547d902cb
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 81319 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 81329 (sshd)
      Tasks: 1 (limit: 2237)
     Memory: 2.2M (peak: 3M)
        CPU: 35ms
   CGroup: /system.slice/ssh.service
           └─81329 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 19 05:59:53 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Sep 19 05:59:53 kali sshd[81329]: Server listening on 0.0.0.0 port 22.
Sep 19 05:59:53 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Sep 19 05:59:53 kali sshd[81329]: Server listening on :: port 22.
(shani@kali)-[~]
└─$
```

Wazuh Logs Confirming Detected Authentication Failures

```
Sep 19 10:00:50 kali sshd-session[81797]: Failed password for invalid user kali from 127.0.0.1 port 41980 ssh2
Sep 19 10:01:05 kali sshd-session[81963]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:07 kali sshd-session[81963]: Failed password for invalid user kali from 127.0.0.1 port 53112 ssh2
Sep 19 10:01:10 kali sshd-session[82000]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:11 kali sshd-session[82000]: Failed password for invalid user kali from 127.0.0.1 port 51998 ssh2
Sep 19 10:01:14 kali sshd-session[82037]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:16 kali sshd-session[82037]: Failed password for invalid user kali from 127.0.0.1 port 52006 ssh2
Sep 19 10:01:18 kali sshd-session[82082]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:14 kali sshd-session[82037]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:10 kali sshd-session[82000]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:05 kali sshd-session[81963]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:01 kali sshd-session[81918]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:00:56 kali sshd-session[81881]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:00:52 kali sshd-session[81844]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:00:48 kali sshd-session[81797]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1
Sep 19 10:01:20 kali sshd-session[82082]: Failed password for invalid user kali from 127.0.0.1 port 47170 ssh2
Sep 19 10:02:36 kali sudo[82719]: shani : TTY=pts/0 ; PWD=/home/shani ; USER=root ; COMMAND=/usr/bin/grep -i 'authentication failure\\|failed password' /var/ossec/logs/alerts/alerts.log
command: /usr/bin/grep -i 'authentication failure\\|failed password' /var/ossec/logs/alerts/alerts.log
og
shani@shani:~$
```